

Matplotlib Basics Explained

 chatgpt.com/c/6a623ade-4d64-83e8-b059-97eda0016762

You said:

matplotlib all concept and properties and function explain with code and very easy word

ChatGPT said:

Matplotlib: Basics to Advanced in Easy Words

Matplotlib is a Python library used to create graphs and charts. It is commonly used with NumPy and Pandas for data analysis and machine learning.

1. Installation and Import

```
pip install matplotlib
```

```
import matplotlib.pyplot as plt  
import numpy as np
```

`plt` is a short name for `matplotlib.pyplot`.

2. Basic Line Graph

```
import matplotlib.pyplot as plt  
  
x = [1, 2, 3, 4, 5]  
y = [10, 20, 15, 30, 25]  
  
plt.plot(x, y)  
  
plt.xlabel("Day")  
plt.ylabel("Sales")  
plt.title("Daily Sales")  
  
plt.show()
```

Important functions:

Function	Meaning
<code>plt.plot()</code>	Create a line graph
<code>plt.xlabel()</code>	Add x-axis name
<code>plt.ylabel()</code>	Add y-axis name
<code>plt.title()</code>	Add chart title
<code>plt.show()</code>	Display the chart

3. Structure of a Matplotlib Chart

Matplotlib has four important parts:

Figure → Complete drawing area
Axes → Actual chart area
Axis → X-axis and Y-axis
Artist → Everything drawn, such as line, text and legend

Example:

```
fig, ax = plt.subplots()

ax.plot([1, 2, 3], [10, 20, 15])
ax.set_title("Simple Graph")
ax.set_xlabel("X Values")
ax.set_ylabel("Y Values")

plt.show()
```

- `fig` means complete canvas.
- `ax` means one graph inside the canvas.
- `plt.subplots()` creates a figure and axes.

The `fig, ax` method is called the object-oriented method. It is recommended for professional projects.

4. Line Graph Properties

```
x = [1, 2, 3, 4, 5]
y = [10, 18, 14, 25, 22]

plt.plot(
    x,
    y,
    color="blue",
    linestyle="--",
    linewidth=3,
    marker="o",
    markersize=8,
    markerfacecolor="yellow",
    markeredgecolor="red",
    alpha=0.8,
    label="Sales"
)

plt.legend()
plt.show()
```

Important line properties

Property	Meaning	Example
color	Line color	"red", "blue", "#12A4D9"
linestyle	Line design	"-", "--", ":", "-."
linewidth	Line thickness	2, 3.5
marker	Point symbol	"o", "s", "^"
markersize	Marker size	8
markerfacecolor	Inside marker color	"yellow"
markeredgecolor	Marker border color	"black"
alpha	Transparency	0.0 to 1.0
label	Name used in legend	"Sales"

Property	Meaning	Example
zorder	Which item appears on top	1, 2, 3

Common markers

Marker	Shape
"o"	Circle
"s"	Square
"^"	Up triangle
"v"	Down triangle
"*"	Star
"D"	Diamond
"x"	Cross
"+"	Plus
."	Small dot

5. Multiple Lines

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales_2025 = [20, 25, 30, 28, 40]
sales_2026 = [25, 30, 27, 35, 45]

plt.plot(months, sales_2025, marker="o", label="2025")
plt.plot(months, sales_2026, marker="s", label="2026")

plt.title("Yearly Sales Comparison")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend()
plt.grid(True)

plt.show()
```


6. Title and Label Properties

```
plt.plot([1, 2, 3], [10, 20, 15])

plt.title(
    "Sales Report",
    fontsize=18,
    color="darkblue",
    fontweight="bold",
    loc="center",
    pad=15
)

plt.xlabel("Month", fontsize=12, color="green")
plt.ylabel("Sales", fontsize=12, color="red")

plt.show()
```

Important text properties:

Property	Meaning
fontsize	Text size
color	Text color
fontweight	"normal" or "bold"
fontstyle	"normal" or "italic"
loc	"left", "center" or "right"
pad	Gap between title and chart
rotation	Rotate text
fontfamily	Text font family

7. Grid

```
plt.plot([1, 2, 3, 4], [10, 30, 20, 40])

plt.grid(
    True,
    axis="both",
    color="gray",
    linestyle="--",
    linewidth=0.7,
    alpha=0.5
)

plt.show()
```

Grid properties:

- `axis="x"`: Only vertical grid lines.
- `axis="y"`: Only horizontal grid lines.
- `axis="both"`: Both grid lines.

8. Legend

```
plt.plot([1, 2, 3], [10, 20, 30], label="Product A")
plt.plot([1, 2, 3], [15, 18, 35], label="Product B")

plt.legend(
    loc="upper left",
    fontsize=11,
    title="Products",
    frameon=True,
    shadow=True,
    ncol=1
)

plt.show()
```

Common `loc` values:

```
best  
upper left  
upper right  
lower left  
lower right  
center  
center left  
center right
```

9. Axis Limits

```
plt.plot([1, 2, 3, 4], [10, 20, 30, 40])  
  
plt.xlim(0, 5)  
plt.ylim(0, 50)  
  
plt.show()
```

Object-oriented form:

```
fig, ax = plt.subplots()  
  
ax.plot([1, 2, 3, 4], [10, 20, 30, 40])  
ax.set_xlim(0, 5)  
ax.set_ylim(0, 50)  
  
plt.show()
```

10. Ticks and Tick Labels

Ticks are the numbers or names displayed on an axis.

```
x = [1, 2, 3, 4]
y = [10, 20, 15, 30]

plt.plot(x, y)

plt.xticks(
    ticks=[1, 2, 3, 4],
    labels=["Jan", "Feb", "Mar", "Apr"],
    rotation=45
)

plt.yticks([0, 10, 20, 30, 40])

plt.show()
```

Change tick appearance:

```
fig, ax = plt.subplots()

ax.plot([1, 2, 3], [10, 20, 15])

ax.tick_params(
    axis="both",
    labelsize=11,
    labelcolor="blue",
    direction="out",
    length=6,
    width=2
)

plt.show()
```

11. Scatter Plot

A scatter plot shows the relationship between two numerical columns.

```
hours = [1, 2, 3, 4, 5, 6]
marks = [40, 50, 55, 65, 75, 85]

plt.scatter(
    hours,
    marks,
    color="purple",
    marker="o",
    s=100,
    alpha=0.7,
    edgecolor="black"
)

plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Study Hours vs Marks")

plt.show()
```

Scatter properties:

Property	Meaning
s	Point size
c or color	Point color
marker	Point shape
alpha	Transparency
edgecolor	Point-border color
linewidth	Border thickness

Property	Meaning
----------	---------

<code>cmap</code>	Color theme
-------------------	-------------

Scatter plot with color map

```
x = [1, 2, 3, 4, 5]
y = [20, 35, 30, 45, 50]
values = [10, 20, 30, 40, 50]
sizes = [100, 200, 300, 400, 500]
```

```
plt.scatter(
    x,
    y,
    c=values,
    s=sizes,
    cmap="viridis",
    alpha=0.8
)
```

```
plt.colorbar(label="Value")
plt.show()
```

12. Vertical Bar Chart

```
products = ["Laptop", "Phone", "Tablet", "Watch"]
sales = [40, 70, 30, 50]
```

```
plt.bar(
    products,
    sales,
    color=["blue", "green", "orange", "purple"],
    width=0.7,
    edgecolor="black"
)
```

```
plt.xlabel("Products")
plt.ylabel("Sales")
plt.title("Product Sales")
```

```
plt.show()
```

Bar properties:

Property	Meaning
color	Bar color
width	Bar width
edgecolor	Border color
linewidth	Border thickness
alpha	Transparency
label	Legend label
bottom	Starting height

13. Horizontal Bar Chart

```
skills = ["Python", "SQL", "ML", "Power BI"]
scores = [90, 85, 80, 75]

plt.barh(skills, scores, color="teal")

plt.xlabel("Skill Score")
plt.title("Technical Skills")

plt.show()
```

14. Add Values on Bars

```
products = ["A", "B", "C", "D"]
sales = [40, 70, 30, 50]

fig, ax = plt.subplots()

bars = ax.bar(products, sales, color="skyblue")
ax.bar_label(bars, padding=3)

ax.set_ylim(0, 80)
ax.set_title("Sales with Values")

plt.show()
```

15. Grouped Bar Chart

```
import numpy as np
import matplotlib.pyplot as plt

departments = ["IT", "HR", "Finance"]
men = [30, 15, 20]
women = [25, 20, 18]

x = np.arange(len(departments))
width = 0.35

plt.bar(x - width / 2, men, width, label="Men")
plt.bar(x + width / 2, women, width, label="Women")

plt.xticks(x, departments)
plt.ylabel("Employees")
plt.title("Employees by Department")
plt.legend()

plt.show()
```


16. Stacked Bar Chart

```
departments = ["IT", "HR", "Finance"]
men = [30, 15, 20]
women = [25, 20, 18]

plt.bar(departments, men, label="Men")
plt.bar(departments, women, bottom=men, label="Women")

plt.ylabel("Employees")
plt.title("Stacked Employee Chart")
plt.legend()

plt.show()
```

`bottom=men` tells Matplotlib to place the women bars above the men bars.

17. Histogram

A histogram shows how numerical data is distributed.

```
marks = [45, 50, 52, 55, 58, 60, 60, 65, 70, 72, 75, 80, 85, 90]

plt.hist(
    marks,
    bins=5,
    color="orange",
    edgecolor="black",
    alpha=0.8
)

plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.title("Marks Distribution")

plt.show()
```

Histogram properties:

Property	Meaning
<code>bins</code>	Number of groups
<code>range</code>	Data range

Property	Meaning
density=True	Show probability density
cumulative=True	Show cumulative count
orientation	"vertical" or "horizontal"
histtype	"bar", "step" or "stepfilled"

18. Pie Chart

```
labels = ["Python", "SQL", "Machine Learning", "Power BI"]
sizes = [35, 25, 25, 15]

plt.pie(
    sizes,
    labels=labels,
    autopct="%1.1f%%",
    startangle=90,
    explode=[0.1, 0, 0, 0],
    shadow=True,
    colors=["gold", "skyblue", "lightgreen", "pink"]
)

plt.title("Skill Distribution")
plt.axis("equal")
plt.show()
```

Pie-chart properties:

Property	Meaning
labels	Category names
autopct	Display percentage
explode	Move a slice outside
startangle	Starting angle
shadow	Add shadow
colors	Slice colors

Property	Meaning
wedgeprops	Slice-border properties

Donut chart

```
plt.pie(
    [40, 30, 20, 10],
    labels=["A", "B", "C", "D"],
    autopct="%1.0f%%",
    wedgeprops={"width": 0.4, "edgecolor": "white"}
)

plt.show()
```

19. Box Plot

A box plot helps find median, spread and outliers.

```
salary = [25000, 28000, 30000, 32000, 35000, 38000, 40000, 90000]

plt.boxplot(
    salary,
    vert=True,
    patch_artist=True,
    showmeans=True,
    boxprops={"facecolor": "lightblue"},
    medianprops={"color": "red", "linewidth": 2}
)

plt.ylabel("Salary")
plt.title("Salary Box Plot")

plt.show()
```

Important parts:

- Middle line: median
- Box: middle 50% of data
- Whiskers: normal minimum and maximum range
- Separate points: possible outliers

20. Violin Plot

A violin plot shows both distribution shape and data spread.

```
data = [  
    [20, 25, 30, 35, 40],  
    [30, 32, 35, 38, 45]  
]  
  
plt.violinplot(  
    data,  
    showmeans=True,  
    showmedians=True  
)  
  
plt.xticks([1, 2], ["Group A", "Group B"])  
plt.ylabel("Score")  
plt.show()
```

21. Area Plot

```
days = [1, 2, 3, 4, 5]  
sales = [10, 20, 15, 30, 25]  
  
plt.fill_between(  
    days,  
    sales,  
    color="skyblue",  
    alpha=0.5  
)  
  
plt.plot(days, sales, color="blue")  
plt.xlabel("Day")  
plt.ylabel("Sales")  
plt.show()
```

22. Stack Plot

```
days = [1, 2, 3, 4, 5]
product_a = [10, 15, 20, 25, 30]
product_b = [5, 10, 12, 15, 18]

plt.stackplot(
    days,
    product_a,
    product_b,
    labels=["Product A", "Product B"],
    colors=["skyblue", "lightgreen"]
)

plt.legend(loc="upper left")
plt.show()
```

23. Step Plot

A step plot changes values in steps instead of using smooth lines.

```
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 30, 25]

plt.step(x, y, where="mid", marker="o")
plt.show()
```

where can be "pre", "mid" or "post".

24. Stem Plot

A stem plot draws a vertical line from the base to every value.

```
x = [1, 2, 3, 4, 5]
y = [5, 12, 8, 15, 10]

plt.stem(x, y)
plt.show()
```

25. Error-Bar Plot

Error bars show uncertainty or possible variation.

```
x = [1, 2, 3, 4]
y = [10, 20, 15, 25]
error = [1, 2, 1.5, 3]
```

```
plt.errorbar(
    x,
    y,
    yerr=error,
    fmt="o-",
    capsize=5,
    color="blue",
    ecolor="red"
)
```

```
plt.show()
```

- **xerr**: Horizontal error.
 - **yerr**: Vertical error.
 - **capsize**: Width of error-bar ends.
 - **fmt**: Marker and line format.
-

26. Heatmap

```
import numpy as np
import matplotlib.pyplot as plt

data = np.array([
    [10, 20, 30],
    [25, 35, 45],
    [40, 50, 60]
])

fig, ax = plt.subplots()

image = ax.imshow(data, cmap="YlGnBu")

for row in range(data.shape[0]):
    for column in range(data.shape[1]):
        ax.text(
            column,
            row,
            data[row, column],
            ha="center",
            va="center",
            color="black"
        )

fig.colorbar(image, ax=ax, label="Value")
ax.set_title("Simple Heatmap")

plt.show()
```

Common color maps:

```
viridis
plasma
inferno
magma
coolwarm
Blues
Greens
Reds
YlGnBu
```

27. Images with Matplotlib

```
import matplotlib.pyplot as plt

image = plt.imread("photo.jpg")

plt.imshow(image)
plt.axis("off")
plt.show()
```

For grayscale:

```
plt.imshow(image, cmap="gray")
plt.axis("off")
plt.show()
```

28. Subplots

Subplots display multiple charts inside one figure.

```
fig, axes = plt.subplots(2, 2, figsize=(10, 7))

axes[0, 0].plot([1, 2, 3], [10, 20, 15])
axes[0, 0].set_title("Line Chart")

axes[0, 1].bar(["A", "B", "C"], [20, 30, 25])
axes[0, 1].set_title("Bar Chart")

axes[1, 0].scatter([1, 2, 3], [30, 20, 40])
axes[1, 0].set_title("Scatter Plot")

axes[1, 1].hist([10, 15, 15, 20, 25, 30], bins=4)
axes[1, 1].set_title("Histogram")

plt.tight_layout()
plt.show()
```

`plt.tight_layout()` automatically fixes spacing and overlap.

29. Figure Properties

```
fig, ax = plt.subplots(
    figsize=(8, 5),
    dpi=100,
    facecolor="lightgray"
)

ax.plot([1, 2, 3], [10, 20, 15])
plt.show()
```

Property	Meaning
<code>figsize=(width, height)</code>	Figure size in inches
<code>dpi</code>	Image quality
<code>facecolor</code>	Figure background
<code>edgecolor</code>	Figure-border color
<code>layout="constrained"</code>	Automatically manage spacing

Example:

```
fig, axes = plt.subplots(
    2,
    2,
    figsize=(10, 7),
    layout="constrained"
)
```

30. Axes Background and Spines

Spines are the borders around the graph.

```
fig, ax = plt.subplots()

ax.plot([1, 2, 3], [10, 20, 15])
ax.set_facecolor("#f2f2f2")

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

ax.spines["left"].set_color("blue")
ax.spines["left"].set_linewidth(2)

plt.show()
```

31. Add Text

```
plt.plot([1, 2, 3, 4], [10, 20, 35, 25])

plt.text(
    3,
    35,
    "Highest Value",
    fontsize=12,
    color="red",
    ha="center"
)

plt.show()
```

Important text alignment properties:

- **ha**: Horizontal alignment
- **va**: Vertical alignment
- **rotation**: Text rotation

- **bbox**: Box around text

```
plt.text(
    2,
    20,
    "Important",
    bbox={
        "facecolor": "yellow",
        "edgecolor": "black",
        "boxstyle": "round"
    }
)
```

32. Annotation with Arrow

```
x = [1, 2, 3, 4, 5]
y = [10, 20, 35, 25, 30]

plt.plot(x, y, marker="o")

plt.annotate(
    "Maximum",
    xy=(3, 35),
    xytext=(3.5, 42),
    arrowprops={
        "facecolor": "black",
        "shrink": 0.05
    }
)

plt.ylim(0, 50)
plt.show()
```

- **xy**: Point being explained.
- **xytext**: Position of annotation text.
- **arrowprops**: Arrow design.

33. Logarithmic Scale

Use a log scale when values have a very large difference.

```
x = [1, 2, 3, 4]
y = [10, 100, 1000, 10000]

plt.plot(x, y, marker="o")
plt.yscale("log")

plt.show()
```

Common scales:

```
plt.xscale("linear")
plt.xscale("log")
plt.yscale("linear")
plt.yscale("log")
```

34. Two Y-Axes

Use two y-axes when two measurements have different scales.

```
months = ["Jan", "Feb", "Mar", "Apr"]
sales = [100, 150, 120, 180]
temperature = [20, 25, 30, 35]

fig, ax1 = plt.subplots()

ax1.plot(months, sales, color="blue", marker="o")
ax1.set_ylabel("Sales", color="blue")

ax2 = ax1.twinx()
ax2.plot(months, temperature, color="red", marker="s")
ax2.set_ylabel("Temperature", color="red")

plt.title("Sales and Temperature")
plt.show()
```

35. Date and Time Plot

```
from datetime import datetime
import matplotlib.pyplot as plt

dates = [
    datetime(2026, 1, 1),
    datetime(2026, 2, 1),
    datetime(2026, 3, 1),
    datetime(2026, 4, 1)
]

sales = [100, 120, 110, 150]

fig, ax = plt.subplots()

ax.plot(dates, sales, marker="o")
fig.autofmt_xdate()

ax.set_title("Monthly Sales")
plt.show()
```

36. 3D Plot

```
import matplotlib.pyplot as plt
import numpy as np

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")

x = np.array([1, 2, 3, 4])
y = np.array([2, 3, 4, 5])
z = np.array([10, 20, 15, 30])

ax.scatter(x, y, z, color="red", s=80)

ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
ax.set_title("3D Scatter Plot")

plt.show()
```

3D surface plot

```
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)

X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

surface = ax.plot_surface(X, Y, Z, cmap="viridis")
fig.colorbar(surface, ax=ax, shrink=0.6)

plt.show()
```

37. Plot Pandas Data

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr"],
    "Sales": [100, 140, 120, 180],
    "Profit": [20, 35, 25, 50]
})

df.plot(
    x="Month",
    y=["Sales", "Profit"],
    kind="line",
    marker="o"
)

plt.title("Sales and Profit")
plt.ylabel("Amount")
plt.grid(True)
plt.show()
```

Common Pandas plot types:

```
df.plot(kind="line")
df.plot(kind="bar")
df.plot(kind="barh")
df.plot(kind="hist")
df.plot(kind="box")
df.plot(kind="area")
df.plot(kind="scatter", x="Sales", y="Profit")
df.plot(kind="pie", y="Sales")
```

38. Chart Styles

```
import matplotlib.pyplot as plt

print(plt.style.available)
```

Apply a style:

```
plt.style.use("ggplot")

plt.plot([1, 2, 3], [10, 20, 15])
plt.show()
```

Apply temporarily:

```
with plt.style.context("dark_background"):
    plt.plot([1, 2, 3], [10, 20, 15])
plt.show()
```

39. Save a Chart

```
plt.plot([1, 2, 3], [10, 20, 15])
plt.title("Sales Chart")

plt.savefig(
    "sales_chart.png",
    dpi=300,
    bbox_inches="tight",
    transparent=False,
    facecolor="white"
)

plt.show()
```

Common formats:

```
plt.savefig("chart.png")
plt.savefig("chart.jpg")
plt.savefig("chart.pdf")
plt.savefig("chart.svg")
```

Important properties:

Property	Meaning
<code>dpi</code>	Image quality
<code>bbox_inches="tight"</code>	Remove unnecessary outer space
<code>transparent=True</code>	Transparent background
<code>facecolor</code>	Saved-image background
<code>pad_inches</code>	Space around saved chart

Call `savefig()` before `show()` for safer results.

40. Clear and Close Figures

```
plt.clf()    # Clear the current figure
plt.cla()    # Clear the current axes
plt.close()  # Close the current figure
plt.close("all") # Close every figure
```

These functions are useful when generating many graphs in a loop.

41. Animation

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

x = np.linspace(0, 2 * np.pi, 200)

fig, ax = plt.subplots()
line, = ax.plot(x, np.sin(x))

ax.set_ylim(-1.2, 1.2)

def update(frame):
    line.set_ydata(np.sin(x + frame / 10))
    return line,

animation = FuncAnimation(
    fig,
    update,
    frames=100,
    interval=50
)

plt.show()
```

- **frames**: Number of animation steps.
 - **interval**: Delay between frames in milliseconds.
 - **update()**: Changes the chart in every frame.
-

42. Most Important **pyplot** Functions

Function	Use
<code>plt.figure()</code>	Create a figure
<code>plt.subplots()</code>	Create figure and axes
<code>plt.plot()</code>	Line chart
<code>plt.scatter()</code>	Scatter chart
<code>plt.bar()</code>	Vertical bar chart
<code>plt.barh()</code>	Horizontal bar chart
<code>plt.hist()</code>	Histogram
<code>plt.pie()</code>	Pie chart
<code>plt.boxplot()</code>	Box plot
<code>plt.violinplot()</code>	Violin plot
<code>plt.fill_between()</code>	Area under line
<code>plt.stackplot()</code>	Stacked area chart
<code>plt.step()</code>	Step chart
<code>plt.stem()</code>	Stem chart
<code>plt.errorbar()</code>	Chart with error
<code>plt.imshow()</code>	Display image or matrix
<code>plt.colorbar()</code>	Add color scale
<code>plt.title()</code>	Add title
<code>plt.xlabel()</code>	Add x-axis label
<code>plt.ylabel()</code>	Add y-axis label
<code>plt.legend()</code>	Add legend
<code>plt.grid()</code>	Add grid

Function	Use
<code>plt.xlim()</code>	Set x-axis range
<code>plt.ylim()</code>	Set y-axis range
<code>plt.xticks()</code>	Change x-axis ticks
<code>plt.yticks()</code>	Change y-axis ticks
<code>plt.text()</code>	Add text
<code>plt.annotate()</code>	Add text with arrow
<code>plt.tight_layout()</code>	Fix subplot spacing
<code>plt.savefig()</code>	Save chart
<code>plt.show()</code>	Display chart

43. `plt` Method vs `ax` Method

Pyplot method	Object-oriented method
<code>plt.plot()</code>	<code>ax.plot()</code>
<code>plt.title()</code>	<code>ax.set_title()</code>
<code>plt.xlabel()</code>	<code>ax.set_xlabel()</code>
<code>plt.ylabel()</code>	<code>ax.set_ylabel()</code>
<code>plt.xlim()</code>	<code>ax.set_xlim()</code>
<code>plt.ylim()</code>	<code>ax.set_ylim()</code>
<code>plt.xticks()</code>	<code>ax.set_xticks()</code>
<code>plt.grid()</code>	<code>ax.grid()</code>
<code>plt.legend()</code>	<code>ax.legend()</code>

Professional format:

```
fig, ax = plt.subplots(figsize=(8, 5))

ax.plot(
    [1, 2, 3, 4],
    [10, 25, 18, 35],
    color="blue",
    marker="o",
    label="Sales"
)

ax.set(
    title="Monthly Sales",
    xlabel="Month",
    ylabel="Sales",
    xlim=(0, 5),
    ylim=(0, 40)
)

ax.grid(alpha=0.3)
ax.legend()

fig.tight_layout()
plt.show()
```

44. Common Errors

Graph does not appear

Use:

```
plt.show()
```

Labels overlap

Use:

```
plt.tight_layout()
```

or:

```
plt.xticks(rotation=45)
```

Old graphs appear together

Create a new figure or close the old one:

```
plt.figure()
```

```
plt.close()
```

X and Y lengths are different

Wrong:

```
x = [1, 2, 3]
y = [10, 20]

plt.plot(x, y)
```

Both lists must contain the same number of values.

Legend does not appear

Add **label** and call **legend()**:

```
plt.plot(x, y, label="Sales")
plt.legend()
```

45. Complete Practical Example

```
import numpy as np
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = np.array([100, 140, 130, 180, 200, 230])
profit = np.array([20, 35, 30, 50, 65, 80])

fig, axes = plt.subplots(
    1,
    2,
    figsize=(12, 5),
    layout="constrained"
)

# Line chart
axes[0].plot(
    months,
    sales,
    color="royalblue",
    marker="o",
    linewidth=2.5,
    label="Sales"
)

axes[0].plot(
    months,
    profit,
    color="green",
    marker="s",
    linewidth=2.5,
    label="Profit"
)

axes[0].set_title("Sales and Profit Trend")
axes[0].set_xlabel("Month")
axes[0].set_ylabel("Amount")
axes[0].grid(alpha=0.3)
axes[0].legend()

# Bar chart
bars = axes[1].bar(
    months,
    sales,
    color="orange",
    edgecolor="black"
)

axes[1].bar_label(bars, padding=3)
```

```
axes[1].set_title("Monthly Sales")
axes[1].set_xlabel("Month")
axes[1].set_ylabel("Sales")
axes[1].set_ylim(0, 260)

fig.suptitle(
    "Company Performance Dashboard",
    fontsize=16,
    fontweight="bold"
)

plt.show()
```

Recommended Learning Order

1. Installation and import
2. Figure, Axes and Axis
3. Line plot
4. Titles, labels, ticks and grid
5. Colors, markers and line styles
6. Legend and annotation
7. Scatter plot
8. Bar and horizontal bar charts
9. Histogram
10. Pie and donut charts
11. Box and violin plots
12. Area and stack plots
13. Heatmaps and images
14. Subplots
15. Object-oriented plotting
16. Dates and logarithmic scales
17. 3D plots
18. Pandas integration
19. Saving charts
20. Animation and customization

The most important interview point is: **Figure** is the complete canvas, while **Axes** is the actual chart placed inside that canvas.

You said:

all types graph and its properties

ChatGPT said:

Matplotlib: All Main Graph Types and Their Properties

Matplotlib contains many specialized plots. Below are the main graph types used in data analysis, machine learning, projects and interviews.

Common setup

```
import numpy as np
import matplotlib.pyplot as plt
```

A. Trend and Comparison Graphs

1. Line Graph

Use a line graph to show changes over time.

```
x = [1, 2, 3, 4, 5]
y = [10, 18, 15, 25, 30]

plt.plot(
    x,
    y,
    color="blue",
    linestyle="--",
    linewidth=2,
    marker="o",
    markersize=8,
    label="Sales",
    alpha=0.8
)

plt.title("Sales Trend")
plt.xlabel("Day")
plt.ylabel("Sales")
plt.grid(True)
plt.legend()
plt.show()
```

Important properties:

Property	Meaning
color	Line color

Property	Meaning
linestyle	"-", "--", ":", "-."
linewidth	Line thickness
marker	Point shape
markersize	Point size
markerfacecolor	Inside-marker color
markeredgecolor	Marker-border color
alpha	Transparency from 0 to 1
label	Name shown in legend
zorder	Controls which item appears on top

2. Multiple-Line Graph

Use it to compare multiple trends.

```
months = ["Jan", "Feb", "Mar", "Apr"]
sales = [100, 130, 120, 170]
profit = [20, 30, 25, 45]

plt.plot(months, sales, marker="o", label="Sales")
plt.plot(months, profit, marker="s", label="Profit")

plt.title("Sales and Profit")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Each line can have its own color, marker, label and line style.

3. Step Plot

Use a step plot when a value changes suddenly, such as stock level or electricity rate.

```
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 30, 25]

plt.step(
    x,
    y,
    where="mid",
    color="purple",
    linewidth=2,
    marker="o"
)

plt.show()
```

Properties:

Property	Meaning
where="pre"	Change before the point
where="mid"	Change between points
where="post"	Change after the point
color	Step-line color
linewidth	Line thickness
marker	Point shape

4. Stem Plot

A stem plot draws vertical lines from the baseline to each value.

```
x = [1, 2, 3, 4, 5]
y = [5, 12, 8, 15, 10]

plt.stem(
    x,
    y,
    linefmt="b-",
    markerfmt="ro",
    basefmt="k-",
    bottom=0
)

plt.show()
```

Properties:

Property	Meaning
<code>linefmt</code>	Stem-line format
<code>markerfmt</code>	Marker format
<code>basefmt</code>	Baseline format
<code>bottom</code>	Baseline position

B. Bar Graphs

5. Vertical Bar Graph

Use it to compare categories.

```
products = ["Laptop", "Phone", "Tablet", "Watch"]
sales = [40, 70, 30, 50]

plt.bar(
    products,
    sales,
    color="skyblue",
    width=0.7,
    edgecolor="black",
    linewidth=1.5,
    alpha=0.8
)

plt.xlabel("Products")
plt.ylabel("Sales")
plt.title("Product Sales")
plt.show()
```

Properties:

Property	Meaning
color	Bar color
width	Bar width
edgecolor	Border color
linewidth	Border thickness
alpha	Transparency
bottom	Starting position
align	"center" or "edge"
label	Legend name

6. Horizontal Bar Graph

Useful when category names are long.

```
skills = ["Python", "Machine Learning", "SQL", "Power BI"]
scores = [90, 80, 85, 75]

plt.barh(
    skills,
    scores,
    color="teal",
    height=0.6,
    edgecolor="black"
)

plt.xlabel("Score")
plt.title("Technical Skills")
plt.show()
```

Special property:

- **height**: Controls horizontal-bar thickness.
- **left**: Controls the starting position.

7. Grouped Bar Graph

Use it to compare two or more groups side by side.

```
departments = ["IT", "HR", "Finance"]  
men = [30, 15, 20]  
women = [25, 20, 18]
```

```
x = np.arange(len(departments))  
width = 0.35
```

```
plt.bar(  
    x - width / 2,  
    men,  
    width=width,  
    label="Men",  
    color="steelblue"  
)
```

```
plt.bar(  
    x + width / 2,  
    women,  
    width=width,  
    label="Women",  
    color="orange"  
)
```

```
plt.xticks(x, departments)  
plt.ylabel("Employees")  
plt.legend()  
plt.show()
```

Important idea: change the x-position of each group so the bars do not overlap.

8. Stacked Bar Graph

Use it to show total value and the contribution of each group.

```
departments = ["IT", "HR", "Finance"]
men = np.array([30, 15, 20])
women = np.array([25, 20, 18])

plt.bar(departments, men, label="Men")
plt.bar(
    departments,
    women,
    bottom=men,
    label="Women"
)

plt.ylabel("Employees")
plt.legend()
plt.show()
```

Important property:

```
bottom=men
```

It places the second group above the first group.

9. 100% Stacked Bar Graph

It shows percentages instead of actual totals.

```
men = np.array([30, 15, 20])
women = np.array([25, 20, 18])

total = men + women

men_percentage = men / total * 100
women_percentage = women / total * 100

plt.bar(departments, men_percentage, label="Men")
plt.bar(
    departments,
    women_percentage,
    bottom=men_percentage,
    label="Women"
)

plt.ylabel("Percentage")
plt.ylim(0, 100)
plt.legend()
plt.show()
```

10. Add Values on Bars

```
fig, ax = plt.subplots()

bars = ax.bar(
    ["A", "B", "C"],
    [40, 70, 50],
    color="lightgreen"
)

ax.bar_label(
    bars,
    fmt="%.0f",
    padding=3,
    color="black",
    fontsize=11
)

ax.set_ylim(0, 80)
plt.show()
```

fmt controls the number format.

C. Relationship Graphs

11. Scatter Plot

Use it to find the relationship between two numerical columns.

```
hours = [1, 2, 3, 4, 5, 6]
marks = [40, 50, 55, 65, 75, 85]

plt.scatter(
    hours,
    marks,
    s=100,
    color="purple",
    marker="o",
    alpha=0.7,
    edgecolor="black",
    linewidth=1
)

plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.show()
```

Properties:

Property	Meaning
s	Point size
c or color	Point color
marker	Point shape
alpha	Transparency
edgecolor	Point-border color
linewidth	Border thickness
cmap	Color theme
vmin, vmax	Minimum and maximum color values

12. Bubble Chart

A bubble chart is a scatter plot where bubble size represents another value.

```
sales = [100, 150, 200, 250, 300]
profit = [20, 35, 45, 60, 80]
customers = [100, 250, 400, 600, 900]

plt.scatter(
    sales,
    profit,
    s=customers,
    c=customers,
    cmap="viridis",
    alpha=0.6,
    edgecolor="black"
)

plt.xlabel("Sales")
plt.ylabel("Profit")
plt.colorbar(label="Customers")
plt.show()
```

- X-axis: sales
- Y-axis: profit
- Bubble size: customers
- Bubble color: customers

13. Hexbin Plot

Useful when a scatter plot contains thousands of overlapping points.

```
np.random.seed(10)

x = np.random.randn(5000)
y = x * 0.7 + np.random.randn(5000)

plt.hexbin(
    x,
    y,
    gridsize=30,
    cmap="Blues",
    mincnt=1
)

plt.colorbar(label="Number of Points")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

Properties:

Property	Meaning
<code>gridsize</code>	Number of hexagon groups
<code>cmap</code>	Color theme
<code>mincnt</code>	Minimum points required
<code>bins="log"</code>	Use logarithmic counts
<code>linewidths</code>	Hexagon-border thickness

14. Two-Dimensional Histogram

It divides both x and y values into rectangular groups.

```
x = np.random.randn(2000)
y = x + np.random.randn(2000)

plt.hist2d(
    x,
    y,
    bins=30,
    cmap="plasma"
)

plt.colorbar(label="Count")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

Properties:

- **bins**: Number of groups.
- **range**: X and Y data range.
- **cmap**: Color theme.
- **density=True**: Shows density instead of count.

D. Distribution Graphs

15. Histogram

Use it to understand numerical-data distribution.

```
marks = [
    40, 45, 50, 52, 55, 58, 60,
    60, 65, 70, 75, 80, 85, 90
]

plt.hist(
    marks,
    bins=5,
    color="orange",
    edgecolor="black",
    alpha=0.8
)

plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.show()
```

Properties:

Property	Meaning
<code>bins</code>	Number of data groups
<code>range</code>	Data range
<code>density=True</code>	Show probability density
<code>cumulative=True</code>	Show running total
<code>histtype</code>	"bar", "step" or "stepfilled"
<code>orientation</code>	"vertical" or "horizontal"
<code>rwidth</code>	Bar width
<code>stacked=True</code>	Stack multiple distributions

16. Multiple Histograms

```
group_a = np.random.normal(60, 10, 200)
group_b = np.random.normal(70, 8, 200)

plt.hist(
    group_a,
    bins=15,
    alpha=0.6,
    label="Group A"
)

plt.hist(
    group_b,
    bins=15,
    alpha=0.6,
    label="Group B"
)

plt.legend()
plt.show()
```

17. Box Plot

Use it to find median, data spread and outliers.

```
salary = [
    25000, 28000, 30000, 32000,
    35000, 38000, 40000, 90000
]

plt.boxplot(
    salary,
    vert=True,
    patch_artist=True,
    showmeans=True,
    showfliers=True,
    boxprops={"facecolor": "lightblue"},
    medianprops={"color": "red", "linewidth": 2},
    meanprops={"marker": "D", "markerfacecolor": "green"},
    whiskerprops={"color": "blue"},
    capprops={"color": "black"},
    flierprops={"marker": "o", "markerfacecolor": "orange"}
)

plt.ylabel("Salary")
plt.show()
```

Properties:

Property	Meaning
vert	Vertical or horizontal
patch_artist	Allow box color
showmeans	Display mean
showfliers	Display outliers
notch	Add median notch
widths	Box width
labels or tick_labels	Group names
boxprops	Box properties
medianprops	Median-line properties
meanprops	Mean properties
whiskerprops	Whisker properties

Property	Meaning
<code>flierprops</code>	Outlier properties

18. Multiple Box Plots

```
python_marks = [60, 70, 75, 80, 90]
sql_marks = [55, 65, 70, 75, 85]
ml_marks = [50, 60, 72, 78, 88]

plt.boxplot(
    [python_marks, sql_marks, ml_marks],
    tick_labels=["Python", "SQL", "ML"],
    patch_artist=True
)

plt.ylabel("Marks")
plt.show()
```

19. Violin Plot

It shows distribution shape, density and spread.

```
data = [
    np.random.normal(60, 10, 100),
    np.random.normal(75, 8, 100)
]

parts = plt.violinplot(
    data,
    showmeans=True,
    showmedians=True,
    showextrema=True,
    widths=0.8
)

plt.xticks([1, 2], ["Group A", "Group B"])
plt.show()
```

Properties:

Property	Meaning
<code>showmeans</code>	Show mean

Property	Meaning
<code>showmedians</code>	Show median
<code>showextrema</code>	Show minimum and maximum
<code>widths</code>	Violin width
<code>vert</code>	Vertical or horizontal
<code>points</code>	Smoothness of violin

20. Error-Bar Graph

It shows the expected error or uncertainty around values.

```
x = [1, 2, 3, 4]
y = [10, 20, 15, 25]
error = [1, 2, 1.5, 3]
```

```
plt.errorbar(
    x,
    y,
    yerr=error,
    fmt="o-",
    color="blue",
    ecolor="red",
    elinewidth=2,
    capsize=5,
    capthick=2
)

plt.show()
```

Properties:

Property	Meaning
<code>xerr</code>	Horizontal error
<code>yerr</code>	Vertical error
<code>fmt</code>	Line and marker format
<code>ecolor</code>	Error-line color

Property	Meaning
<code>elinewidth</code>	Error-line thickness
<code>capsize</code>	Error-cap size
<code>capthick</code>	Error-cap thickness

E. Composition Graphs

21. Pie Chart

Use it to show parts of a total.

```
labels = ["Python", "SQL", "ML", "Power BI"]
sizes = [35, 25, 25, 15]

plt.pie(
    sizes,
    labels=labels,
    colors=["gold", "skyblue", "lightgreen", "pink"],
    autopct="%1.1f%%",
    startangle=90,
    explode=[0.1, 0, 0, 0],
    shadow=True
)

plt.title("Skill Distribution")
plt.axis("equal")
plt.show()
```

Properties:

Property	Meaning
<code>labels</code>	Slice names
<code>colors</code>	Slice colors
<code>autopct</code>	Percentage format
<code>explode</code>	Move slices outward
<code>startangle</code>	Starting angle

Property	Meaning
<code>shadow</code>	Add shadow
<code>counterclock</code>	Direction
<code>labeldistance</code>	Label distance
<code>pctdistance</code>	Percentage distance
<code>wedgeprops</code>	Slice-border properties
<code>textprops</code>	Text properties

22. Donut Chart

A donut chart is a pie chart with an empty center.

```
plt.pie(
    [40, 30, 20, 10],
    labels=["A", "B", "C", "D"],
    autopct="%1.0f%%",
    wedgeprops={
        "width": 0.4,
        "edgecolor": "white",
        "linewidth": 2
    }
)

plt.show()
```

The `width` property creates the center hole.

23. Nested Donut Chart

It displays two related levels.

```

outer = [60, 40]
inner = [35, 25, 20, 20]

plt.pie(
    outer,
    radius=1,
    labels=["Technical", "Other"],
    wedgeprops={"width": 0.3, "edgecolor": "white"}
)

plt.pie(
    inner,
    radius=0.7,
    labels=["Python", "ML", "Communication", "Management"],
    wedgeprops={"width": 0.3, "edgecolor": "white"}
)

plt.show()

```

24. Area Graph

It shows the area below a line.

```

days = [1, 2, 3, 4, 5]
sales = [10, 20, 15, 30, 25]

plt.fill_between(
    days,
    sales,
    color="skyblue",
    alpha=0.5,
    label="Sales"
)

plt.plot(days, sales, color="blue")
plt.legend()
plt.show()

```

Properties:

Property	Meaning
<code>y1, y2</code>	Area boundaries
<code>where</code>	Fill only when a condition is true

Property	Meaning
<code>interpolate</code>	Improve boundary crossing
<code>step</code>	Fill using steps
<code>color</code>	Area color
<code>alpha</code>	Transparency

25. Stacked Area Graph

It shows how multiple values contribute to a total over time.

```

days = [1, 2, 3, 4, 5]
product_a = [10, 15, 20, 25, 30]
product_b = [5, 10, 12, 15, 18]

plt.stackplot(
    days,
    product_a,
    product_b,
    labels=["Product A", "Product B"],
    colors=["skyblue", "lightgreen"],
    alpha=0.8,
    baseline="zero"
)

plt.legend(loc="upper left")
plt.show()

```

Properties:

- `labels`: Area names.
- `colors`: Area colors.
- `alpha`: Transparency.
- `baseline`: "zero", "sym", "wiggle" or "weighted_wiggle".

F. Matrix and Density Graphs

26. Heatmap

A heatmap represents values using colors.

```
data = np.array([
    [10, 20, 30],
    [25, 35, 45],
    [40, 50, 60]
])

fig, ax = plt.subplots()

image = ax.imshow(
    data,
    cmap="YlGnBu",
    interpolation="nearest",
    aspect="auto",
    vmin=0,
    vmax=60
)

for row in range(data.shape[0]):
    for column in range(data.shape[1]):
        ax.text(
            column,
            row,
            data[row, column],
            ha="center",
            va="center"
        )

fig.colorbar(image, ax=ax, label="Value")
plt.show()
```

Properties:

Property	Meaning
cmap	Color theme
vmin, vmax	Color-value range
interpolation	Pixel-smoothing method
aspect	Image shape
origin	"upper" or "lower"
extent	Axis boundaries
alpha	Transparency

27. Matrix Plot

```
matrix = np.random.randint(1, 100, size=(5, 5))

plt.matshow(
    matrix,
    cmap="coolwarm"
)

plt.colorbar()
plt.show()
```

`matshow()` is a simple function for displaying matrices.

28. Pseudocolor Mesh

Useful for displaying values over an x-y grid.

```
x = np.linspace(0, 5, 20)
y = np.linspace(0, 5, 20)

X, Y = np.meshgrid(x, y)
Z = np.sin(X) * np.cos(Y)

plt.pcolormesh(
    X,
    Y,
    Z,
    cmap="viridis",
    shading="auto"
)

plt.colorbar(label="Z Value")
plt.show()
```

Properties:

Property	Meaning
<code>cmap</code>	Color map
<code>shading</code>	"auto", "flat", "nearest"
<code>edgecolors</code>	Cell-border color

Property	Meaning
linewidth	Cell-border thickness
vmin, vmax	Color range

G. Contour and Surface Graphs

29. Contour Plot

A contour plot connects locations having the same value.

```
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)

X, Y = np.meshgrid(x, y)
Z = X**2 + Y**2

contours = plt.contour(
    X,
    Y,
    Z,
    levels=10,
    colors="black",
    linewidths=1
)

plt.clabel(contours, inline=True, fontsize=8)
plt.show()
```

Properties:

Property	Meaning
levels	Number or positions of contour levels
colors	Line colors
cmap	Color map
linewidths	Line thickness
linestyles	Contour-line style

Property	Meaning
----------	---------

alpha	Transparency
-------	--------------

30. Filled Contour Plot

```
filled = plt.contourf(
    X,
    Y,
    Z,
    levels=20,
    cmap="viridis",
    alpha=0.9
)

plt.colorbar(filled)
plt.show()
```

`contourf()` fills the spaces between contour lines.

H. Vector and Direction Graphs

31. Quiver Plot

A quiver plot uses arrows to show direction and strength.


```
x, y = np.meshgrid(
    np.arange(0, 5),
    np.arange(0, 5)
)

u = np.ones_like(x)
v = y

plt.quiver(
    x,
    y,
    u,
    v,
    color="blue",
    scale=20,
    width=0.008
)

plt.show()
```

Properties:

Property	Meaning
color	Arrow color
scale	Arrow-length scaling
width	Arrow width
headwidth	Arrow-head width
headlength	Arrow-head length
pivot	Arrow position: "tail", "middle", "tip"

32. Stream Plot

A stream plot shows the flow of a vector field.

```
y, x = np.mgrid[-3:3:100j, -3:3:100j]

u = -y
v = x

speed = np.sqrt(u**2 + v**2)

plt.streamplot(
    x,
    y,
    u,
    v,
    color=speed,
    cmap="viridis",
    density=1.5,
    linewidth=1
)

plt.colorbar(label="Speed")
plt.show()
```

Properties:

Property	Meaning
density	Number of flow lines
color	Line color or values
cmap	Color theme
linewidth	Line thickness
arrowsize	Arrow size
arrowstyle	Arrow shape
start_points	Starting positions

I. Time and Schedule Graphs

33. Date-Time Graph

```
from datetime import datetime

dates = [
    datetime(2026, 1, 1),
    datetime(2026, 2, 1),
    datetime(2026, 3, 1),
    datetime(2026, 4, 1)
]

sales = [100, 130, 120, 170]

fig, ax = plt.subplots()

ax.plot_date(
    dates,
    sales,
    fmt="o-",
    color="blue"
)

fig.autofmt_xdate()
plt.show()
```

Useful properties:

- **fmt**: Marker and line format.
- **xdate**: Treat x-axis as dates.
- **ydate**: Treat y-axis as dates.
- **fig.autofmt_xdate()**: Fix overlapping date labels.

34. Broken Bar Graph or Simple Gantt Chart

Use it to display tasks over time.

```
fig, ax = plt.subplots()

ax.broken_barh(
    [(1, 3), (6, 2)],
    (10, 8),
    facecolors="tab:blue"
)

ax.broken_barh(
    [(2, 4)],
    (22, 8),
    facecolors="tab:orange"
)

ax.set_xlim(0, 10)
ax.set_ylim(5, 35)

ax.set_yticks([14, 26])
ax.set_yticklabels(["Task A", "Task B"])

ax.set_xlabel("Days")
ax.grid(True)

plt.show()
```

Properties:

Property	Meaning
First tuple	Start position and duration
Second tuple	Vertical position and height
facecolors	Task color
edgecolors	Border color
linewidth	Border thickness

35. Event Plot

It shows when events happened.

```
events = [  
    [1, 2, 4, 7],  
    [1.5, 3, 5, 8],  
    [2, 4, 6, 9]  
]  
  
plt.eventplot(  
    events,  
    orientation="horizontal",  
    colors=["red", "green", "blue"],  
    lineoffsets=[1, 2, 3],  
    linelengths=0.8  
)  
  
plt.xlabel("Time")  
plt.show()
```

Properties:

- **orientation**: "horizontal" or "vertical".
- **colors**: Event colors.
- **lineoffsets**: Group positions.
- **linelengths**: Event-line sizes.
- **linewidths**: Event-line thickness.

J. Polar and Circular Graphs

36. Polar Graph

```
theta = np.linspace(0, 2 * np.pi, 100)  
radius = 1 + np.sin(4 * theta)  
  
fig, ax = plt.subplots(  
    subplot_kw={"projection": "polar"}  
)  
  
ax.plot(  
    theta,  
    radius,  
    color="purple",  
    linewidth=2  
)  
  
ax.fill(theta, radius, alpha=0.3)  
plt.show()
```

Polar axes properties:

```
ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)
ax.set_rmax(2)
ax.set_rticks([0.5, 1, 1.5, 2])
```

37. Radar or Spider Chart

Use it to compare several skill scores.

```
skills = ["Python", "SQL", "ML", "Power BI", "Communication"]
scores = [90, 85, 80, 75, 70]

angles = np.linspace(
    0,
    2 * np.pi,
    len(skills),
    endpoint=False
).tolist()

scores += scores[:1]
angles += angles[:1]

fig, ax = plt.subplots(
    figsize=(6, 6),
    subplot_kw={"projection": "polar"}
)

ax.plot(angles, scores, marker="o", linewidth=2)
ax.fill(angles, scores, alpha=0.25)

ax.set_xticks(angles[:-1])
ax.set_xticklabels(skills)
ax.set_ylim(0, 100)

plt.show()
```

Important points:

- Repeat the first angle and first value to close the shape.
- `set_ylim()` controls the score range.
- `fill()` adds color inside the radar graph.

K. Three-Dimensional Graphs

38. 3D Line Graph

```
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

t = np.linspace(0, 10, 100)

x = np.sin(t)
y = np.cos(t)
z = t

ax.plot(
    x,
    y,
    z,
    color="blue",
    linewidth=2
)

ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")

plt.show()
```

39. 3D Scatter Plot

```
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

x = np.random.rand(50)
y = np.random.rand(50)
z = np.random.rand(50)

points = ax.scatter(
    x,
    y,
    z,
    c=z,
    s=80,
    cmap="viridis",
    alpha=0.8
)

fig.colorbar(points, ax=ax, shrink=0.6)
plt.show()
```

Properties:

- **s**: Point size.
 - **c**: Point color values.
 - **cmap**: Color map.
 - **depthshade**: Add depth shading.
 - **alpha**: Transparency.
-

40. 3D Surface Plot

```
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)

X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

surface = ax.plot_surface(
    X,
    Y,
    Z,
    cmap="viridis",
    edgecolor="none",
    alpha=0.9
)

fig.colorbar(surface, ax=ax, shrink=0.6)
plt.show()
```

Properties:

Property	Meaning
<code>cmap</code>	Surface colors
<code>edgecolor</code>	Grid-border color
<code>alpha</code>	Transparency
<code>rstride</code>	Row step
<code>cstride</code>	Column step
<code>linewidth</code>	Grid thickness
<code>antialiased</code>	Smooth edges

41. 3D Wireframe Plot

```
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

ax.plot_wireframe(
    X,
    Y,
    Z,
    color="blue",
    linewidth=0.7,
    rstride=5,
    cstride=5
)

plt.show()
```

A wireframe displays only the grid lines of a surface.

42. 3D Bar Graph

```
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

x = np.array([1, 2, 3, 4])
y = np.zeros(4)
z = np.zeros(4)

dx = np.full(4, 0.5)
dy = np.full(4, 0.5)
dz = np.array([10, 20, 15, 30])

ax.bar3d(
    x,
    y,
    z,
    dx,
    dy,
    dz,
    color="orange",
    shade=True
)

plt.show()
```

Properties:

- `dx`: Bar width on x-axis.
- `dy`: Bar depth on y-axis.
- `dz`: Bar height.
- `color`: Bar color.
- `shade`: Add light and shadow.

Common Properties for Almost Every Graph

Property	Meaning
<code>color</code>	Main color
<code>alpha</code>	Transparency
<code>label</code>	Legend name
<code>zorder</code>	Drawing order
<code>visible</code>	Show or hide an object
<code>linewidth</code>	Line or border thickness
<code>linestyle</code>	Line design
<code>marker</code>	Point shape
<code>edgecolor</code>	Border color
<code>facecolor</code>	Inside color
<code>cmap</code>	Color theme

Property Meaning

`clip_on` Keep drawing inside axes

Common Chart-Formatting Functions

```
fig, ax = plt.subplots(figsize=(8, 5), dpi=120)
```

```
ax.set_title(  
    "Chart Title",  
    fontsize=16,  
    fontweight="bold",  
    color="darkblue",  
    pad=15  
)
```

```
ax.set_xlabel("X-Axis", fontsize=12)  
ax.set_ylabel("Y-Axis", fontsize=12)
```

```
ax.set_xlim(0, 10)  
ax.set_ylim(0, 100)
```

```
ax.grid(  
    True,  
    axis="both",  
    linestyle="--",  
    alpha=0.4  
)
```

```
ax.legend(  
    loc="best",  
    fontsize=10,  
    frameon=True,  
    shadow=True  
)
```

```
ax.tick_params(  
    axis="both",  
    labelsize=10,  
    rotation=0,  
    direction="out"  
)
```

```
plt.tight_layout()  
plt.show()
```

Which Graph Should You Use?

Requirement	Best graph
Show a trend over time	Line graph
Compare categories	Bar graph
Compare groups	Grouped bar
Show parts of total	Pie or donut
Show contribution over time	Stacked area
Find relationship	Scatter plot
Add a third numerical variable	Bubble chart
Show numerical distribution	Histogram
Find median and outliers	Box plot
Show distribution shape	Violin plot
Show correlation matrix	Heatmap
Show many overlapping points	Hexbin
Show uncertainty	Error-bar plot
Show direction and flow	Quiver or stream plot
Show geographical-height-like values	Contour plot
Compare multiple skills	Radar chart
Show three numerical dimensions	3D scatter
Show a mathematical surface	3D surface plot
Show project schedule	Broken bar or Gantt chart

You said:

advanced graph and project of matplotlib

ChatGPT said:

Advanced Matplotlib Graphs and Complete Projects

Advanced Matplotlib is not only about creating different graphs. It also includes:

- Combining multiple graphs
- Adding confidence ranges
- Using multiple axes
- Creating dashboards
- Adding annotations
- Creating heatmaps and 3D graphs
- Displaying ML-model results
- Creating animations
- Saving professional reports

Required libraries

```
pip install matplotlib numpy pandas scikit-learn
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

1. Line Graph with Confidence Range

The line shows the average value. The shaded area shows possible variation.

```

x = np.arange(1, 13)

average_sales = np.array([
    100, 120, 130, 125, 150, 170,
    180, 175, 190, 210, 220, 240
])

error = np.array([
    10, 12, 8, 15, 10, 13,
    12, 14, 10, 15, 12, 18
])

fig, ax = plt.subplots(figsize=(9, 5))

ax.plot(
    x,
    average_sales,
    color="blue",
    marker="o",
    linewidth=2.5,
    label="Average Sales"
)

ax.fill_between(
    x,
    average_sales - error,
    average_sales + error,
    color="blue",
    alpha=0.2,
    label="Possible Range"
)

ax.set_title("Monthly Sales with Confidence Range")
ax.set_xlabel("Month")
ax.set_ylabel("Sales")
ax.set_xticks(x)
ax.grid(alpha=0.3)
ax.legend()

plt.show()

```

Important properties:

Property	Meaning
<code>fill_between()</code>	Fill area between two values
<code>alpha</code>	Transparency

Property	Meaning
<code>y1, y2</code>	Lower and upper ranges
<code>where</code>	Fill only when a condition is true
<code>interpolate</code>	Smoothly handle crossing lines

2. Dual Y-Axis Graph

Use it when two columns have different measurement scales.


```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]

sales = [100, 130, 125, 170, 190, 220]
temperature = [18, 21, 25, 30, 34, 36]

fig, ax1 = plt.subplots(figsize=(9, 5))

ax1.plot(
    months,
    sales,
    color="blue",
    marker="o",
    linewidth=2
)

ax1.set_xlabel("Month")
ax1.set_ylabel("Sales", color="blue")
ax1.tick_params(axis="y", labelcolor="blue")

ax2 = ax1.twinx()

ax2.plot(
    months,
    temperature,
    color="red",
    marker="s",
    linestyle="--",
    linewidth=2
)

ax2.set_ylabel("Temperature °C", color="red")
ax2.tick_params(axis="y", labelcolor="red")

plt.title("Sales and Temperature")
fig.tight_layout()
plt.show()
```

`ax1.twinx()` creates another y-axis on the right side.

3. Advanced Correlation Heatmap

A correlation value is between **-1** and **1**.

- Near **1**: strong positive relationship
- Near **-1**: strong negative relationship

- Near 0: weak relationship

```
np.random.seed(42)

data = pd.DataFrame({
    "Rainfall": np.random.randint(400, 1200, 100),
    "Temperature": np.random.randint(18, 40, 100),
    "Humidity": np.random.randint(40, 95, 100),
    "Fertilizer": np.random.randint(50, 250, 100),
    "Crop Yield": np.random.uniform(1.5, 6.5, 100)
})

correlation = data.corr()

fig, ax = plt.subplots(figsize=(8, 6))

image = ax.imshow(
    correlation,
    cmap="coolwarm",
    vmin=-1,
    vmax=1
)

ax.set_xticks(range(len(correlation.columns)))
ax.set_yticks(range(len(correlation.columns)))

ax.set_xticklabels(correlation.columns, rotation=45, ha="right")
ax.set_yticklabels(correlation.columns)

for row in range(len(correlation)):
    for column in range(len(correlation)):
        value = correlation.iloc[row, column]

        ax.text(
            column,
            row,
            f"{value:.2f}",
            ha="center",
            va="center",
            color="black"
        )

fig.colorbar(image, ax=ax, label="Correlation")
ax.set_title("Feature Correlation Heatmap")

plt.tight_layout()
plt.show()
```

4. Hexbin Density Graph

Use a hexbin graph when a scatter plot has thousands of overlapping points.

```
np.random.seed(42)

x = np.random.normal(50, 15, 5000)
y = 0.8 * x + np.random.normal(0, 10, 5000)

fig, ax = plt.subplots(figsize=(8, 6))

graph = ax.hexbin(
    x,
    y,
    gridsize=35,
    cmap="viridis",
    mincnt=1,
    linewidths=0.2
)

fig.colorbar(graph, ax=ax, label="Number of Points")

ax.set_title("Customer Income vs Spending")
ax.set_xlabel("Income")
ax.set_ylabel("Spending")

plt.show()
```

Important properties:

Property	Meaning
<code>gridsize</code>	Number of hexagonal groups
<code>mincnt</code>	Minimum required points
<code>cmap</code>	Color theme
<code>bins="log"</code>	Use logarithmic color calculation
<code>linewidths</code>	Hexagon-border size

5. Contour Graph

A contour graph joins positions having the same value.

```
x = np.linspace(-5, 5, 150)
y = np.linspace(-5, 5, 150)

X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

fig, ax = plt.subplots(figsize=(8, 6))

filled = ax.contourf(
    X,
    Y,
    Z,
    levels=30,
    cmap="viridis"
)

lines = ax.contour(
    X,
    Y,
    Z,
    levels=10,
    colors="black",
    linewidths=0.5
)

ax.clabel(lines, inline=True, fontsize=8)

fig.colorbar(filled, ax=ax, label="Z Value")
ax.set_title("Filled Contour Graph")

plt.show()
```

Properties:

- **levels**: Number of height levels
- **cmap**: Color map
- **linewidths**: Contour-line thickness
- **linestyles**: Contour-line design
- **clabel()**: Display values on contour lines

6. Three-Dimensional Surface Graph

```
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)

X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection="3d")

surface = ax.plot_surface(
    X,
    Y,
    Z,
    cmap="plasma",
    edgecolor="none",
    alpha=0.9,
    antialiased=True
)

ax.set_title("3D Surface Graph")
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")

fig.colorbar(
    surface,
    ax=ax,
    shrink=0.6,
    label="Height"
)

plt.show()
```

Important properties:

Property	Meaning
<code>cmap</code>	Surface colors
<code>edgecolor</code>	Grid-border color
<code>alpha</code>	Transparency
<code>rstride</code>	Row step

Property	Meaning
<code>cstride</code>	Column step
<code>antialiased</code>	Smooth edges
<code>shade</code>	Light and shadow effect

7. Radar Graph

A radar graph is useful for displaying technical skills or model comparisons.

```
skills = [  
    "Python",  
    "SQL",  
    "Machine Learning",  
    "Statistics",  
    "Power BI"  
]  
  
scores = [90, 85, 82, 75, 78]  
  
angles = np.linspace(  
    0,  
    2 * np.pi,  
    len(skills),  
    endpoint=False  
).tolist()  
  
scores = scores + scores[:1]  
angles = angles + angles[:1]  
  
fig, ax = plt.subplots(  
    figsize=(7, 7),  
    subplot_kw={"projection": "polar"}  
)  
  
ax.plot(  
    angles,  
    scores,  
    color="blue",  
    linewidth=2,  
    marker="o"  
)  
  
ax.fill(  
    angles,  
    scores,  
    color="skyblue",  
    alpha=0.4  
)  
  
ax.set_xticks(angles[:-1])  
ax.set_xticklabels(skills)  
  
ax.set_ylim(0, 100)  
ax.set_yticks([20, 40, 60, 80, 100])  
  
ax.set_title(  
    "AI/ML Engineer Skill Analysis",  
    fontsize=15,  
    pad=20
```

```
)  
  
plt.show()
```

8. Sankey Diagram

A Sankey diagram shows how one total value moves into different categories.

```
import matplotlib.pyplot as plt  
from matplotlib.sankey import Sankey  
  
fig, ax = plt.subplots(figsize=(9, 6))  
  
sankey = Sankey(  
    ax=ax,  
    unit=" units",  
    scale=0.01,  
    offset=0.2  
)  
  
sankey.add(  
    flows=[100, -55, -25, -20],  
    labels=[  
        "Total Production",  
        "Market",  
        "Storage",  
        "Waste"  
    ],  
    orientations=[0, 1, -1, 0],  
    facecolor="lightgreen",  
    edgecolor="black"  
)  
  
sankey.finish()  
  
ax.set_title("Agricultural Production Flow")  
plt.show()
```

The total of **flows** must be zero:

$$100 - 55 - 25 - 20 = 0$$

9. Stream Flow Graph

It shows the direction and speed of movement.


```
y, x = np.mgrid[-3:3:100j, -3:3:100j]

u = -y
v = x

speed = np.sqrt(u**2 + v**2)

fig, ax = plt.subplots(figsize=(8, 6))

stream = ax.streamplot(
    x,
    y,
    u,
    v,
    color=speed,
    cmap="viridis",
    density=1.5,
    linewidth=1.2,
    arrowsize=1.5
)

fig.colorbar(
    stream.lines,
    ax=ax,
    label="Flow Speed"
)

ax.set_title("Wind Flow Visualization")
plt.show()
```

10. Animated Graph

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

x = np.linspace(0, 2 * np.pi, 200)

fig, ax = plt.subplots(figsize=(8, 5))

line, = ax.plot(
    x,
    np.sin(x),
    color="blue",
    linewidth=2
)

ax.set_xlim(0, 2 * np.pi)
ax.set_ylim(-1.2, 1.2)
ax.grid(alpha=0.3)

def update(frame):
    new_y = np.sin(x + frame / 10)
    line.set_ydata(new_y)
    return line,

animation = FuncAnimation(
    fig,
    update,
    frames=100,
    interval=50,
    blit=True
)

plt.show()
```

Animation properties:

Property	Meaning
frames	Number of animation steps
interval	Delay in milliseconds
blit	Update only changed parts
repeat	Repeat animation

Property	Meaning
<code>repeat_delay</code>	Delay before repeating

Save the animation:

```
animation.save(  
    "animation.gif",  
    writer="pillow",  
    fps=20  
)
```

Install Pillow if required:

```
pip install pillow
```

Project 1: Crop Yield Analysis Dashboard

This project is useful for your AI/ML crop-yield prediction portfolio.

It displays:

- Average yield by year
- Average yield by crop
- Rainfall and crop-yield relationship
- Yield distribution
- Crop comparison

- Correlation heatmap

Complete project code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# -----
# 1. Create sample agricultural data
# -----

np.random.seed(42)

number_of_records = 300

crops = np.random.choice(
    ["Rice", "Wheat", "Maize", "Sugarcane"],
    number_of_records
)

years = np.random.choice(
    np.arange(2018, 2026),
    number_of_records
)

rainfall = np.random.normal(
    850,
    180,
    number_of_records
).clip(300, 1400)

temperature = np.random.normal(
    27,
    4,
    number_of_records
).clip(15, 42)

humidity = np.random.normal(
    65,
    12,
    number_of_records
).clip(30, 95)

fertilizer = np.random.normal(
    120,
    30,
    number_of_records
).clip(40, 220)
```

```

crop_effect = {
    "Rice": 0.8,
    "Wheat": 0.4,
    "Maize": 0.6,
    "Sugarcane": 1.1
}

crop_bonus = np.array([
    crop_effect[crop]
    for crop in crops
])

crop_yield = (
    1.5
    + 0.0025 * rainfall
    + 0.008 * fertilizer
    - 0.035 * np.abs(temperature - 26)
    + crop_bonus
    + np.random.normal(0, 0.35, number_of_records)
)

crop_yield = crop_yield.clip(1, None)

df = pd.DataFrame({
    "Year": years,
    "Crop": crops,
    "Rainfall": rainfall,
    "Temperature": temperature,
    "Humidity": humidity,
    "Fertilizer": fertilizer,
    "Yield": crop_yield
})

# -----
# 2. Prepare summary data
# -----

yearly_yield = (
    df.groupby("Year", as_index=False)["Yield"]
    .mean()
    .sort_values("Year")
)

crop_average = (
    df.groupby("Crop")["Yield"]
    .mean()
    .sort_values(ascending=False)
)

correlation_columns = [
    "Rainfall",

```

```

    "Temperature",
    "Humidity",
    "Fertilizer",
    "Yield"
]

correlation = df[correlation_columns].corr()

crop_names = crop_average.index.tolist()

# -----
# 3. Create dashboard
# -----

plt.style.use("seaborn-v0_8-whitegrid")

fig = plt.figure(
    figsize=(16, 11),
    facecolor="#f4f7fb"
)

grid = fig.add_gridspec(
    2,
    3,
    hspace=0.35,
    wspace=0.30
)

ax1 = fig.add_subplot(grid[0, 0])
ax2 = fig.add_subplot(grid[0, 1])
ax3 = fig.add_subplot(grid[0, 2])
ax4 = fig.add_subplot(grid[1, 0])
ax5 = fig.add_subplot(grid[1, 1])
ax6 = fig.add_subplot(grid[1, 2])

fig.suptitle(
    "Crop Yield Analysis Dashboard",
    fontsize=22,
    fontweight="bold",
    color="#17324d"
)

# -----
# Graph 1: Yearly yield trend
# -----

ax1.plot(
    yearly_yield["Year"],
    yearly_yield["Yield"],
    color="#1565c0",
    marker="o",

```

```

        linewidth=2.5
    )

    ax1.fill_between(
        yearly_yield["Year"],
        yearly_yield["Yield"],
        color="#90caf9",
        alpha=0.35
    )

    ax1.set_title("Average Yield by Year")
    ax1.set_xlabel("Year")
    ax1.set_ylabel("Yield: Tons per Hectare")
    ax1.grid(alpha=0.3)

# -----
# Graph 2: Crop average bar chart
# -----

bars = ax2.bar(
    crop_average.index,
    crop_average.values,
    color=[
        "#2e7d32",
        "#66bb6a",
        "#f9a825",
        "#8d6e63"
    ],
    edgecolor="black",
    alpha=0.85
)

ax2.bar_label(
    bars,
    fmt="%.2f",
    padding=3
)

ax2.set_title("Average Yield by Crop")
ax2.set_xlabel("Crop")
ax2.set_ylabel("Average Yield")
ax2.tick_params(axis="x", rotation=20)

# -----
# Graph 3: Rainfall vs yield
# -----

scatter = ax3.scatter(
    df["Rainfall"],
    df["Yield"],
    c=df["Temperature"],

```

```
s=45,
cmap="coolwarm",
alpha=0.7,
edgecolor="white",
linewidth=0.4
)

# Add regression trend line
trend = np.polyfit(
    df["Rainfall"],
    df["Yield"],
    1
)

trend_function = np.poly1d(trend)

sorted_rainfall = np.sort(df["Rainfall"])

ax3.plot(
    sorted_rainfall,
    trend_function(sorted_rainfall),
    color="black",
    linestyle="--",
    linewidth=2,
    label="Trend"
)

ax3.set_title("Rainfall vs Crop Yield")
ax3.set_xlabel("Rainfall in mm")
ax3.set_ylabel("Yield")
ax3.legend()

fig.colorbar(
    scatter,
    ax=ax3,
    label="Temperature °C"
)

# -----
# Graph 4: Yield histogram
# -----

ax4.hist(
    df["Yield"],
    bins=18,
    color="#7e57c2",
    edgecolor="white",
    alpha=0.85
)

mean_yield = df["Yield"].mean()
```



```
ax4.axvline(
    mean_yield,
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"Mean = {mean_yield:.2f}"
)

ax4.set_title("Yield Distribution")
ax4.set_xlabel("Yield")
ax4.set_ylabel("Frequency")
ax4.legend()

# -----
# Graph 5: Crop box plot
# -----

box_data = [
    df.loc[df["Crop"] == crop, "Yield"]
    for crop in crop_names
]

box = ax5.boxplot(
    box_data,
    labels=crop_names,
    patch_artist=True,
    showmeans=True
)

box_colors = [
    "#81c784",
    "#ffcc80",
    "#90caf9",
    "#ce93d8"
]

for patch, color in zip(
    box["boxes"],
    box_colors
):
    patch.set_facecolor(color)

for median in box["medians"]:
    median.set_color("red")
    median.set_linewidth(2)

ax5.set_title("Yield Variation by Crop")
ax5.set_xlabel("Crop")
ax5.set_ylabel("Yield")
ax5.tick_params(axis="x", rotation=20)
```

```
# -----  
# Graph 6: Correlation heatmap  
# -----  
  
heatmap = ax6.imshow(  
    correlation,  
    cmap="coolwarm",  
    vmin=-1,  
    vmax=1  
)  
  
ax6.set_xticks(  
    range(len(correlation.columns))  
)  
  
ax6.set_yticks(  
    range(len(correlation.columns))  
)  
  
ax6.set_xticklabels(  
    correlation.columns,  
    rotation=45,  
    ha="right"  
)  
  
ax6.set_yticklabels(  
    correlation.columns  
)  
  
for row in range(len(correlation)):  
    for column in range(len(correlation)):  
        value = correlation.iloc[row, column]  
  
        ax6.text(  
            column,  
            row,  
            f"{value:.2f}",  
            ha="center",  
            va="center",  
            fontsize=9  
        )  
  
ax6.set_title("Feature Correlation")  
  
fig.colorbar(  
    heatmap,  
    ax=ax6,  
    shrink=0.8  
)
```

```

# -----
# 4. Add dashboard summary
# -----

summary_text = (
    f"Records: {len(df)} | "
    f"Average Yield: {df['Yield'].mean():.2f} | "
    f"Highest Yield: {df['Yield'].max():.2f} | "
    f"Best Crop: {crop_average.index[0]}"
)

fig.text(
    0.5,
    0.94,
    summary_text,
    ha="center",
    fontsize=12,
    color="#455a64"
)

# -----
# 5. Save and show dashboard
# -----

plt.savefig(
    "crop_yield_dashboard.png",
    dpi=300,
    bbox_inches="tight",
    facecolor=fig.get_facecolor()
)

plt.show()

```

How to use your real CSV file

Replace the data-creation section with:

```
df = pd.read_csv("crop_yield_data.csv")
```

Your CSV should contain columns similar to:

Year
Crop
Rainfall
Temperature
Humidity
Fertilizer
Yield

Project 2: Machine-Learning Model Evaluation Dashboard

This project trains a Random Forest model and displays:

- Confusion matrix
- ROC curve
- Precision-recall curve

- Feature importance

Complete code

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    roc_curve,
    roc_auc_score,
    precision_recall_curve,
    average_precision_score
)

# -----
# 1. Create classification data
# -----

X, y = make_classification(
    n_samples=1200,
    n_features=6,
    n_informative=4,
    n_redundant=1,
    class_sep=1.2,
    random_state=42
)

feature_names = [
    "Age",
    "Income",
    "Experience",
    "Credit Score",
    "Transactions",
    "Account Balance"
]

# -----
# 2. Split data
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
```

```
    random_state=42,
    stratify=y
)

# -----
# 3. Train model
# -----

model = RandomForestClassifier(
    n_estimators=200,
    max_depth=8,
    random_state=42
)

model.fit(X_train, y_train)

# -----
# 4. Make predictions
# -----

predictions = model.predict(X_test)
probabilities = model.predict_proba(X_test)[:, 1]

accuracy = accuracy_score(
    y_test,
    predictions
)

matrix = confusion_matrix(
    y_test,
    predictions
)

fpr, tpr, _ = roc_curve(
    y_test,
    probabilities
)

roc_score = roc_auc_score(
    y_test,
    probabilities
)

precision, recall, _ = precision_recall_curve(
    y_test,
    probabilities
)

average_precision = average_precision_score(
    y_test,
    probabilities
```

```
)

importance = model.feature_importances_

# -----
# 5. Create dashboard
# -----

fig, axes = plt.subplots(
    2,
    2,
    figsize=(14, 10),
    layout="constrained"
)

fig.suptitle(
    f"Machine Learning Model Dashboard | Accuracy: {accuracy:.2%}",
    fontsize=18,
    fontweight="bold"
)

# -----
# Confusion matrix
# -----

image = axes[0, 0].imshow(
    matrix,
    cmap="Blues"
)

axes[0, 0].set_title("Confusion Matrix")
axes[0, 0].set_xlabel("Predicted Class")
axes[0, 0].set_ylabel("Actual Class")

axes[0, 0].set_xticks([0, 1])
axes[0, 0].set_yticks([0, 1])

axes[0, 0].set_xticklabels(
    ["Negative", "Positive"]
)

axes[0, 0].set_yticklabels(
    ["Negative", "Positive"]
)

for row in range(2):
    for column in range(2):
        axes[0, 0].text(
            column,
            row,
            matrix[row, column],
```

```

        ha="center",
        va="center",
        fontsize=15,
        fontweight="bold"
    )

fig.colorbar(
    image,
    ax=axes[0, 0]
)

# -----
# ROC curve
# -----

axes[0, 1].plot(
    fpr,
    tpr,
    color="darkorange",
    linewidth=2.5,
    label=f"AUC = {roc_score:.3f}"
)

axes[0, 1].plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    color="gray",
    label="Random Model"
)

axes[0, 1].set_title("ROC Curve")
axes[0, 1].set_xlabel("False Positive Rate")
axes[0, 1].set_ylabel("True Positive Rate")
axes[0, 1].legend()
axes[0, 1].grid(alpha=0.3)

# -----
# Precision-recall curve
# -----

axes[1, 0].plot(
    recall,
    precision,
    color="green",
    linewidth=2.5,
    label=f"AP = {average_precision:.3f}"
)

axes[1, 0].set_title("Precision-Recall Curve")
axes[1, 0].set_xlabel("Recall")

```



```
axes[1, 0].set_ylabel("Precision")
axes[1, 0].legend()
axes[1, 0].grid(alpha=0.3)

# -----
# Feature importance
# -----

sorted_positions = np.argsort(importance)

axes[1, 1].barh(
    np.array(feature_names)[sorted_positions],
    importance[sorted_positions],
    color="purple",
    alpha=0.8
)

axes[1, 1].set_title("Feature Importance")
axes[1, 1].set_xlabel("Importance Score")

plt.savefig(
    "ml_model_dashboard.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

Additional Matplotlib Project Ideas

Project	Graphs to use
Sales dashboard	Line, bar, pie, histogram, heatmap
Stock-market analysis	Line, candlestick, moving averages
Weather dashboard	Dual-axis, area, error band, heatmap
Student-result analysis	Bar, histogram, box plot, radar
Customer segmentation	Scatter, bubble, 3D scatter
Employee attrition analysis	Bar, pie, box plot, heatmap
COVID or disease analysis	Line, area, map, rolling average
House-price analysis	Scatter, hexbin, histogram, box plot

Project	Graphs to use
Crop-yield prediction	Scatter, line, heatmap, box plot
ML-model comparison	Grouped bar, ROC, PR curve, heatmap

A strong Matplotlib portfolio project should include data cleaning, at least four meaningful graphs, proper labels, insights, a dashboard layout and a high-quality saved output.

You said:

churn project major project

ChatGPT said:

Major Project: Customer Churn Prediction System

Customer churn means a customer stops using a company's service.

This project will:

- Analyse why customers leave
- Visualize churn patterns with Matplotlib
- Train multiple machine-learning models
- Compare model performance
- Identify high-risk customers
- Export churn predictions
- Save the trained model

1. Project information

Project title: AI-Based Customer Churn Prediction and Retention System

Problem type: Binary classification

0 → Customer will stay
1 → Customer may leave

Technologies:

- Python
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

- Joblib
- Jupyter Notebook or VS Code

2. Dataset columns

Use the IBM Telco Customer Churn dataset. Save it as:

Telco-Customer-Churn.csv

Important columns include:

Column	Meaning
customerID	Unique customer number
gender	Male or female
SeniorCitizen	Senior citizen or not
Partner	Has a partner
Dependents	Has dependents
tenure	Number of months with company
InternetService	DSL, fiber optic or no service
Contract	Monthly, one-year or two-year
PaymentMethod	Customer payment method
MonthlyCharges	Monthly bill
TotalCharges	Total amount paid

Column	Meaning
Churn	Customer left or stayed

3. Project folder structure

```
customer-churn-project/  
├── data/  
│   └── Telco-Customer-Churn.csv  
├── models/  
│   └── customer_churn_model.pkl  
├── outputs/  
│   ├── churn_eda_dashboard.png  
│   ├── model_dashboard.png  
│   └── high_risk_customers.csv  
├── customer_churn.ipynb  
└── requirements.txt
```

4. Install libraries

```
pip install numpy pandas matplotlib scikit-learn joblib
```

5. Import libraries

```
import warnings
warnings.filterwarnings("ignore")

import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.base import clone
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    precision_recall_curve
)
```

6. Load and understand data

```
df = pd.read_csv(
    "data/Telco-Customer-Churn.csv"
)

print("Dataset shape:", df.shape)
print(df.head())
print(df.info())
print(df.isnull().sum())
print(df.duplicated().sum())
```

Expected meaning:

```
print(df["Churn"].value_counts())  
print(df["Churn"].value_counts(normalize=True) * 100)
```

7. Data cleaning

```
# Remove unnecessary spaces from column names  
df.columns = df.columns.str.strip()  
  
# Convert TotalCharges into a numerical column  
df["TotalCharges"] = pd.to_numeric(  
    df["TotalCharges"],  
    errors="coerce"  
)  
  
# Remove duplicate rows  
df = df.drop_duplicates()  
  
# Remove rows where target is missing  
df = df.dropna(subset=["Churn"])  
  
# Convert target into 0 and 1  
df["Churn"] = (  
    df["Churn"]  
    .astype(str)  
    .str.strip()  
    .map({  
        "No": 0,  
        "Yes": 1  
    })  
)  
  
print(df.isnull().sum())  
print(df["Churn"].value_counts())
```

We do not need to manually fill all missing values. The machine-learning pipeline will handle them.

8. Advanced Churn EDA Dashboard

This dashboard displays:

- Churn percentage
- Churn by contract
- Tenure distribution
- Monthly-charge comparison
- Churn by internet service

- Tenure versus monthly charges

```
plt.style.use("seaborn-v0_8-whitegrid")

fig, axes = plt.subplots(
    2,
    3,
    figsize=(17, 11),
    layout="constrained"
)

fig.suptitle(
    "Customer Churn Analysis Dashboard",
    fontsize=22,
    fontweight="bold",
    color="#17324d"
)

# -----
# Graph 1: Churn percentage
# -----

churn_counts = df["Churn"].value_counts().sort_index()

axes[0, 0].pie(
    churn_counts,
    labels=["Stayed", "Churned"],
    autopct="%1.1f%%",
    startangle=90,
    explode=[0, 0.08],
    colors=["#42a5f5", "#ef5350"],
    shadow=True,
    wedgeprops={
        "edgecolor": "white",
        "linewidth": 2
    }
)

axes[0, 0].set_title("Overall Churn Percentage")

# -----
# Graph 2: Churn by contract
# -----

contract_churn = (
    df.groupby("Contract")["Churn"]
    .mean()
    .mul(100)
    .sort_values()
)
```

```

bars = axes[0, 1].bar(
    contract_churn.index,
    contract_churn.values,
    color=["#66bb6a", "#ffa726", "#ef5350"],
    edgecolor="black",
    alpha=0.85
)

axes[0, 1].bar_label(
    bars,
    fmt="%.1f%%",
    padding=3
)

axes[0, 1].set_title("Churn Rate by Contract")
axes[0, 1].set_xlabel("Contract Type")
axes[0, 1].set_ylabel("Churn Rate (%)")
axes[0, 1].tick_params(axis="x", rotation=15)

# -----
# Graph 3: Tenure distribution
# -----

axes[0, 2].hist(
    df.loc[df["Churn"] == 0, "tenure"],
    bins=20,
    alpha=0.65,
    color="#42a5f5",
    edgecolor="white",
    label="Stayed"
)

axes[0, 2].hist(
    df.loc[df["Churn"] == 1, "tenure"],
    bins=20,
    alpha=0.65,
    color="#ef5350",
    edgecolor="white",
    label="Churned"
)

axes[0, 2].set_title("Tenure Distribution")
axes[0, 2].set_xlabel("Tenure in Months")
axes[0, 2].set_ylabel("Customers")
axes[0, 2].legend()

# -----
# Graph 4: Monthly charges box plot
# -----

```



```

stayed_charges = df.loc[
    df["Churn"] == 0,
    "MonthlyCharges"
]

churned_charges = df.loc[
    df["Churn"] == 1,
    "MonthlyCharges"
]

box = axes[1, 0].boxplot(
    [stayed_charges, churned_charges],
    labels=["Stayed", "Churned"],
    patch_artist=True,
    showmeans=True,
    medianprops={
        "color": "black",
        "linewidth": 2
    }
)

box["boxes"][0].set_facecolor("#90caf9")
box["boxes"][1].set_facecolor("#ef9a9a")

axes[1, 0].set_title("Monthly Charges by Churn")
axes[1, 0].set_ylabel("Monthly Charges")

# -----
# Graph 5: Churn by internet service
# -----

internet_churn = (
    df.groupby("InternetService")["Churn"]
    .mean()
    .mul(100)
    .sort_values()
)

internetBars = axes[1, 1].barh(
    internet_churn.index,
    internet_churn.values,
    color="#7e57c2",
    alpha=0.85
)

axes[1, 1].bar_label(
    internetBars,
    fmt="%.1f%%",
    padding=3
)

```

```
axes[1, 1].set_title("Churn Rate by Internet Service")
axes[1, 1].set_xlabel("Churn Rate (%)")

# -----
# Graph 6: Tenure vs monthly charges
# -----

scatter = axes[1, 2].scatter(
    df["tenure"],
    df["MonthlyCharges"],
    c=df["Churn"],
    cmap="coolwarm",
    s=25,
    alpha=0.55,
    edgecolor="none"
)

axes[1, 2].set_title("Tenure vs Monthly Charges")
axes[1, 2].set_xlabel("Tenure")
axes[1, 2].set_ylabel("Monthly Charges")

colorbar = fig.colorbar(
    scatter,
    ax=axes[1, 2]
)

colorbar.set_ticks([0, 1])
colorbar.set_ticklabels(["Stayed", "Churned"])

plt.savefig(
    "outputs/churn_eda_dashboard.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

9. Prepare data for machine learning

```
# Remove customer ID because it does not help prediction
X = df.drop(
    columns=["customerID", "Churn"]
)

y = df["Churn"]

# Separate numerical and categorical columns
numerical_columns = (
    X.select_dtypes(include=np.number)
    .columns
    .tolist()
)

categorical_columns = (
    X.select_dtypes(exclude=np.number)
    .columns
    .tolist()
)

print("Numerical columns:")
print(numerical_columns)

print("\nCategorical columns:")
print(categorical_columns)
```

10. Create preprocessing pipeline

Numerical preprocessing

- Fill missing values with median
- Scale values to a similar range

Categorical preprocessing

- Fill missing values with the most common value

- Convert text into numbers using one-hot encoding

```
numerical_pipeline = Pipeline(  
    steps=[  
        (  
            "imputer",  
            SimpleImputer(strategy="median")  
        ),  
        (  
            "scaler",  
            StandardScaler()  
        )  
    ]  
)  
  
categorical_pipeline = Pipeline(  
    steps=[  
        (  
            "imputer",  
            SimpleImputer(strategy="most_frequent")  
        ),  
        (  
            "encoder",  
            OneHotEncoder(  
                handle_unknown="ignore"  
            )  
        )  
    ]  
)  
  
preprocessor = ColumnTransformer(  
    transformers=[  
        (  
            "numerical",  
            numerical_pipeline,  
            numerical_columns  
        ),  
        (  
            "categorical",  
            categorical_pipeline,  
            categorical_columns  
        )  
    ]  
)
```

11. Split training and testing data

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)  
  
print("Training data:", X_train.shape)  
print("Testing data:", X_test.shape)
```

stratify=y keeps approximately the same churn percentage in the training and testing data.

12. Train multiple models

```
models = {
    "Logistic Regression": LogisticRegression(
        max_iter=2000,
        class_weight="balanced",
        random_state=42
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=300,
        max_depth=12,
        min_samples_split=5,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    ),

    "Extra Trees": ExtraTreesClassifier(
        n_estimators=300,
        max_depth=14,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    )
}

trained_models = {}
results =

for model_name, estimator in models.items():

    pipeline = Pipeline(
        steps=[
            (
                "preprocessor",
                clone(preprocessor)
            ),
            (
                "model",
                estimator
            )
        ]
    )

    pipeline.fit(
        X_train,
        y_train
    )
```

```
predictions = pipeline.predict(X_test)

probabilities = pipeline.predict_proba(
    X_test
)[: , 1]

results.append({
    "Model": model_name,
    "Accuracy": accuracy_score(
        y_test,
        predictions
    ),
    "Precision": precision_score(
        y_test,
        predictions
    ),
    "Recall": recall_score(
        y_test,
        predictions
    ),
    "F1 Score": f1_score(
        y_test,
        predictions
    ),
    "ROC-AUC": roc_auc_score(
        y_test,
        probabilities
    )
})

trained_models[model_name] = pipeline

results_df = pd.DataFrame(results)

print(
    results_df.sort_values(
        "ROC-AUC",
        ascending=False
    )
)
```

13. Select the best model

```
best_model_name = (
    results_df
    .sort_values(
        "ROC-AUC",
        ascending=False
    )
    .iloc[0]["Model"]
)

best_model = trained_models[
    best_model_name
]

print("Best model:", best_model_name)
```

For churn prediction, recall is especially important because it measures how many actual churn customers the model successfully finds.

14. Detailed model evaluation

```
best_predictions = best_model.predict(
    X_test
)

best_probabilities = best_model.predict_proba(
    X_test
)[: , 1]

print(
    classification_report(
        y_test,
        best_predictions,
        target_names=[
            "Stayed",
            "Churned"
        ]
    )
)
```


Metrics in easy words

Metric	Meaning
Accuracy	Total correct predictions
Precision	How many predicted churners actually churned
Recall	How many actual churners the model found
F1 score	Balance between precision and recall
ROC-AUC	Overall ability to separate churn and non-churn

Metric	Meaning
Confusion matrix	Detailed correct and wrong predictions

15. Model Evaluation Dashboard

```

fig, axes = plt.subplots(
    2,
    2,
    figsize=(15, 11),
    layout="constrained"
)

fig.suptitle(
    f"Churn Model Evaluation: {best_model_name}",
    fontsize=20,
    fontweight="bold"
)

# -----
# Graph 1: Model comparison
# -----

comparison = results_df.set_index("Model")[
    [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score",
        "ROC-AUC"
    ]
]

comparison.plot(
    kind="bar",
    ax=axes[0, 0],
    colormap="viridis",
    edgecolor="black"
)

axes[0, 0].set_title("Model Performance Comparison")
axes[0, 0].set_ylabel("Score")
axes[0, 0].set_ylim(0, 1)
axes[0, 0].tick_params(axis="x", rotation=15)
axes[0, 0].legend(fontsize=8)

# -----
# Graph 2: Confusion matrix
# -----

```

```
matrix = confusion_matrix(
    y_test,
    best_predictions
)

image = axes[0, 1].imshow(
    matrix,
    cmap="Blues"
)

axes[0, 1].set_title("Confusion Matrix")
axes[0, 1].set_xlabel("Predicted Class")
axes[0, 1].set_ylabel("Actual Class")

axes[0, 1].set_xticks([0, 1])
axes[0, 1].set_yticks([0, 1])

axes[0, 1].set_xticklabels([
    "Stayed",
    "Churned"
])

axes[0, 1].set_yticklabels([
    "Stayed",
    "Churned"
])

for row in range(2):
    for column in range(2):
        axes[0, 1].text(
            column,
            row,
            matrix[row, column],
            ha="center",
            va="center",
            fontsize=18,
            fontweight="bold"
        )

fig.colorbar(
    image,
    ax=axes[0, 1]
)

# -----
# Graph 3: ROC curves
# -----

for model_name, model in trained_models.items():
```

```

probabilities = model.predict_proba(
    X_test
)[: , 1]

fpr, tpr, _ = roc_curve(
    y_test,
    probabilities
)

auc_score = roc_auc_score(
    y_test,
    probabilities
)

axes[1, 0].plot(
    fpr,
    tpr,
    linewidth=2,
    label=f"{model_name}: {auc_score:.3f}"
)

axes[1, 0].plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    color="gray",
    label="Random Model"
)

axes[1, 0].set_title("ROC Curve")
axes[1, 0].set_xlabel("False Positive Rate")
axes[1, 0].set_ylabel("True Positive Rate")
axes[1, 0].legend(fontsize=8)
axes[1, 0].grid(alpha=0.3)

# -----
# Graph 4: Precision-recall curve
# -----

precision, recall, thresholds = (
    precision_recall_curve(
        y_test,
        best_probabilities
    )
)

axes[1, 1].plot(
    recall,
    precision,
    color="#2e7d32",
    linewidth=2.5

```

```
)

axes[1, 1].set_title("Precision-Recall Curve")
axes[1, 1].set_xlabel("Recall")
axes[1, 1].set_ylabel("Precision")
axes[1, 1].grid(alpha=0.3)

plt.savefig(
    "outputs/model_dashboard.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

16. Feature Importance Graph

This explains which customer details have the largest effect on churn predictions.

```
processor = best_model.named_steps[
    "preprocessor"
]

final_estimator = best_model.named_steps[
    "model"
]

feature_names = (
    processor.get_feature_names_out()
)

if hasattr(
    final_estimator,
    "feature_importances_"
):
    importance_values = (
        final_estimator.feature_importances_
    )

elif hasattr(
    final_estimator,
    "coef_"
):
    importance_values = np.abs(
        final_estimator.coef_[0]
    )

else:
    importance_values = None

if importance_values is not None:

    feature_importance = pd.DataFrame({
        "Feature": feature_names,
        "Importance": importance_values
    })

    feature_importance = (
        feature_importance
        .sort_values(
            "Importance",
            ascending=False
        )
        .head(15)
        .sort_values("Importance")
    )

    plt.figure(figsize=(10, 7))
```

```
bars = plt.barh(
    feature_importance["Feature"],
    feature_importance["Importance"],
    color="#7e57c2",
    alpha=0.85
)

plt.bar_label(
    bars,
    fmt="%.3f",
    padding=3
)

plt.title(
    f"Top Churn Features: {best_model_name}",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Importance Score")
plt.tight_layout()
plt.show()
```

17. Create Customer Risk Levels

```
risk_customers = X_test.copy()

risk_customers["ActualChurn"] = (
    y_test.values
)

risk_customers["ChurnProbability"] = (
    best_probabilities
)

risk_customers["RiskLevel"] = pd.cut(
    risk_customers["ChurnProbability"],
    bins=[0, 0.40, 0.70, 1.0],
    labels=[
        "Low Risk",
        "Medium Risk",
        "High Risk"
    ],
    include_lowest=True
)

risk_customers = risk_customers.sort_values(
    "ChurnProbability",
    ascending=False
)

print(
    risk_customers[
        [
            "tenure",
            "Contract",
            "MonthlyCharges",
            "TotalCharges",
            "ChurnProbability",
            "RiskLevel"
        ]
    ].head(20)
)
```

Risk levels

0%–40% → Low risk
40%–70% → Medium risk
70%–100% → High risk

18. Export High-Risk Customers

```
high_risk_customers = risk_customers[
    risk_customers["RiskLevel"] == "High Risk"
]

high_risk_customers.to_csv(
    "outputs/high_risk_customers.csv",
    index=False
)

print(
    "High-risk customers:",
    len(high_risk_customers)
)
```

The business team can contact these customers and provide discounts, better plans or support.

19. Save the Model

```
joblib.dump(
    best_model,
    "models/customer_churn_model.pkl"
)

print("Model saved successfully.")
```

Load it later:

```
loaded_model = joblib.load(
    "models/customer_churn_model.pkl"
)
```

20. Predict One Customer

The easiest safe method is to provide a customer record with the same columns used during training.

```
sample_customer = X_test.iloc[[0]]

churn_prediction = loaded_model.predict(
    sample_customer
)[0]

churn_probability = loaded_model.predict_proba(
    sample_customer
)[0, 1]

print(
    "Prediction:",
    "Customer may leave"
    if churn_prediction == 1
    else "Customer may stay"
)

print(
    f"Churn probability: "
    f"{churn_probability:.2%}"
)
```

21. Business Recommendations

After analysing the data, recommendations may include:

- Give long-term contract offers to monthly-contract customers.
- Provide special offers to new customers with low tenure.
- Review high monthly charges.
- Improve fiber-optic service quality.
- Contact high-risk customers before they leave.
- Offer automatic-payment discounts.
- Provide personalised retention plans.

22. Resume Project Description

AI-Based Customer Churn Prediction System

- Developed an end-to-end customer churn prediction system using Python, Pandas, Matplotlib and Scikit-learn.
- Performed data cleaning, exploratory analysis, feature engineering and categorical encoding.
- Compared Logistic Regression, Random Forest and Extra Trees using accuracy, precision, recall, F1 and ROC-AUC.
- Created interactive-style analytical dashboards and identified high-risk customers for targeted retention campaigns.

- Saved the complete preprocessing and prediction pipeline using Joblib.

23. Interview Explanation

I developed a customer churn prediction system to identify customers who may stop using a company's services. I cleaned customer data, performed exploratory analysis with Matplotlib, transformed numerical and categorical features, and trained multiple classification models. I selected the best model using ROC-AUC and recall because identifying actual churn customers is important for the business. Finally, I classified customers into low, medium and high-risk groups so the company could take retention action.